



SATA TECHNOLOGY AND BUSINESS COLLEGE



SOFTWARE ENGINEERING



By MESFIN.M.

CHAPTER ONE

INTRODUCTION

Software Engineering Fundamentals

- ✓ What is software?

- Types of software

- Software Applications

- Characteristics of software

- ✓ What is Software Crisis?

- What is Software Engineering?

- ✓ Professional and Ethical Responsibility

Objectives of this Activity

The objective of this activity is to introduce you to the concept of software engineering. From this activity u will:

- learn the importance of software engineering
- develop a broad understanding of the software engineering domain
- set out the answers to key questions about software engineering
- understand ethical and professional issues

WHAT IS SOFTWARE?

- Software is collection of **computer** programs, procedures, rules, and associated documentations and data .
- This definition clearly states that software is **not just** programs, but includes all the associated **documentation and data**.
- This implies that the discipline dealing with the development of **software should not deal only with developing programs, but with developing all the things that constitute software**.
- Software products may be **developed** for a **particular customer** or may be developed for a general **market**

SOFTWARE TYPES

- There are different types of software. One of the most important types are **custom** software, **generic** software and **embedded** software.
 - ***Custom (Bespoke)*** : is developed to meet the **specific needs of a particular** customer according to their specification
 - Example: **web sites, air traffic control systems,** software for managing the specialized finances of large organizations, etc
 - ***Generic***: is designed to be sold on the **open market**, to perform functions that many people need, Often called COTS (**Commercial Off The Shelf**) or **Shrink-wrapped**.
 - Example: PC software such as word **processors, excel, compilers,** etc

- *Embedded:* Built into hardware
 - It runs specific hardware devices which are typically sold on the open market
 - Hard to change
 - Example: washing machines, DVD players, microwave ovens and automobiles. Unlike generic software, users cannot usually replace embedded software or upgrade it without also replacing the hardware
- New software can be created by developing new programs, configuring generic software systems or reusing existing software.

- An important difference between these types of software is that, in **generic** products, the **organization** that develops the software **controls the software specification**.
- For **custom** products, the specification is usually developed and controlled **by the organization that is buying the software**. The software developers must work to that specification.

WHY SOFTWARE IS IMPORTANT?

- Because it affects nearly every aspect of our **lives** and has become pervasive in our **commerce**, our **culture**, and our **everyday** activities.
- Software is both a **product** and a vehicle for **delivering** a product
- Software **delivers** the most important product of **our time** —**information**.
- Software transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., Internet) and provides the means for acquiring information in all of its forms.

- The economies of **ALL** developed nations are **dependent** on **software**.
- **More** and **more** systems are software **controlled** (**transportation**, medical, telecommunications, military, industrial, entertainment,)
- Expenditure on software represents a significant fraction of **GNP** in all developed countries.
 - **GNP** stands for Gross National Product. **GNP** per capita is the **dollar** value of a country's final output of **goods** and **services** in a year, divided by its population. It reflects the **average income of a country's citizens**.

SOFTWARE COSTS

- Software costs often dominate computer system costs. The costs of **software** on a PC are often **greater** than the **hardware** cost.
- Software costs more to **maintain** than it does to **develop**. For systems with a long life, maintenance costs may be several times development costs.

SOFTWARE APPLICATIONS

- Software may be applied in any **situation** for which a **pre-specified set of procedural steps** has been defined.
- It is somewhat difficult to develop meaningful generic categories for software applications. As software complexity grows, neat **compartmentalization** **disappears**.
- The following software areas indicate the breadth of potential **applications**:
 - **System software**-designed to run a computer's hardware and application **programs**
 - Microsoft windows Os, Ubuntu, etc
 - **Real-time software**- Must react immediately
 - Software that monitors/ analyzes/ controls real-world events as they occur is called real time

- **Business information software**-Used to run businesses
 - **Accuracy** and **security** of data are key concerns
 - Functions: recording sales, managing accounts, printing bills
- **Engineering & scientific software**- Computer-aided design, system simulation, and other interactive applications
- **Embedded software**- control and manage hardware devices
- **Personal application software**
 - Word processor, spreadsheets
- **Communication software**- provide remote access to systems and exchange files and messages

SOFTWARE CHARACTERISTICS

- To gain an understanding of software, it is important to examine the characteristics of software that make it different from other things that human beings build.
 - *Software is developed or engineered*, it is not manufactured in the classical sense.
 - *Software doesn't wear out*, but it does deteriorate.
 - It *deteriorates* by having its design changed: changes often tend to introduce new defects

SOFTWARE CRISIS

- During 1970s, large numbers of software projects failed mostly due to human factors. This problem is referred to as the “**software crisis**”.
- After surveying 8,000 IT projects, Standish Group reported about 30% of all projects were cancelled.
- Software engineering appears to be among the few options available to **tackle** the present software crisis.
- The expenses that organizations all around the world are incurring on software purchases compared to those on hardware purchases have been showing a worrying trend over the years

- Organizations are spending larger and larger portions of their budget on software.
- The cost we are talking of here is not on account of increased features, but due to ineffective development of the product characterized by **inefficient resource usage, time and cost over-runs**.
- There are many factors that have contributed to the making of the present **software crisis**.
- Factors are **larger problem sizes, lack of adequate training in software engineering, increasing skill shortage, and low productivity improvements**.
- This can be solved with **proper use of software engineering principles**

- Software engineering
 - Systematic approach to software development
 - Application of software principles
 - Provides sets of methods, tools, and procedures
 - Ensures consistent software quality

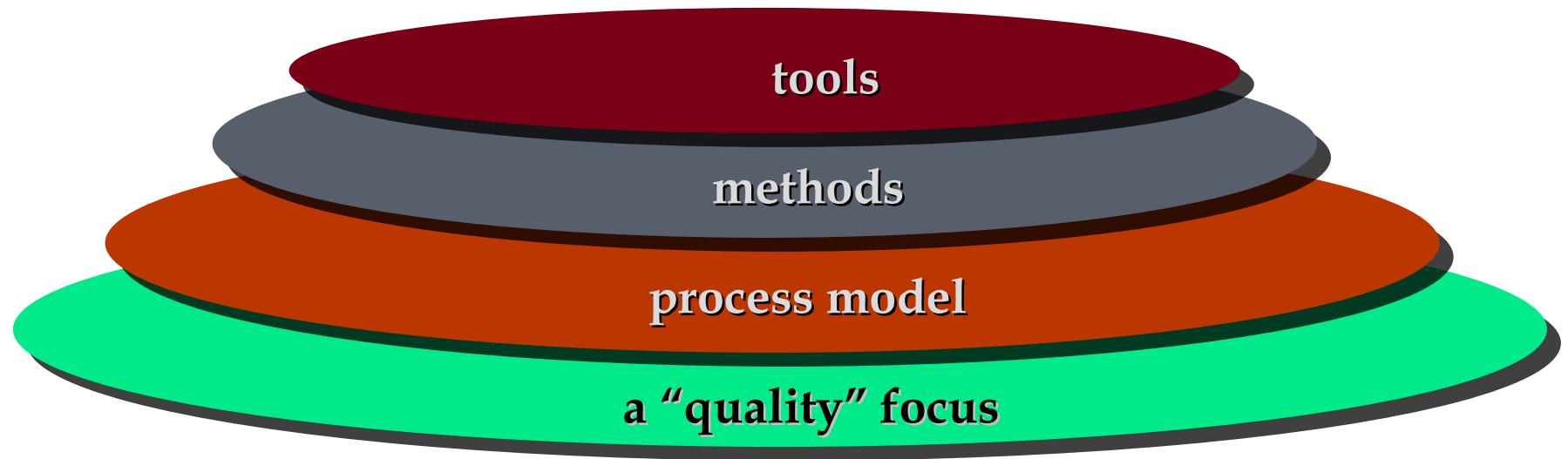
- Many more projects fail because they don't use software engineering. Failures occur because:
 - of the size of the project relative to previous efforts
 - key personnel have left
 - of failure to understand requirements
 - the project delivers, but lacks the required **quality**
 - of the **introduction** of new technology
 - of many, many other reasons
- For this reasons, a software developer have to use a principle of software engineering to deliver quality software products to his/her customers with a consistent, well-managed and cost-effective process

SOFTWARE ENGINEERING

- Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system **specification** through to **maintaining** the system after it has gone into use.
- Engineering discipline
 - Using appropriate **theories** and **methods** to solve problems bearing in mind **organizational** and **financial** constraints.
- All aspects of software production
 - Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

- Software Engineering is about:
 - Quality **improvement**, **Reliability**, **Maintainability**
 - Within the **framework** of:
 - Time , Money, Customer requirements
- Software engineering is concerned with cost-effective software development and also concerned with theories, methods and tools for professional software development

Software Engineering: A layered Technology



- Software Engineering is a **layered technology** (as shown in the above slide) that relies on an **organizational commitment to quality**.
- Any engineering approach must rest on organizational commitment to **quality** which **fosters** a **continuous process improvement culture**.

- Method provides technical **how-to's** for **building** software. It **encompasses** many tasks including **communication, requirement analysis, design modeling, program construction, testing and support.**
- Tools provide **automated or semi-automated support** for the process and methods.

FAQS ABOUT SOFTWARE ENGINEERING

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What is software engineering?	Software engineering is an engineering discipline that is concerned with all aspects of software production.
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
What is the difference between software engineering and computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the difference between software engineering and system engineering?	System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

Question	Answer
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.
What differences has the web made to software engineering?	The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

ESSENTIAL ATTRIBUTES OF GOOD SOFTWARE

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

SOFTWARE PROCESS

- A set of activities whose goal is the development or evolution of software.
- Generic activities in all software processes are:
 - **Specification** - what the system **should do** and its **development** constraints
 - **Development** - production of the **software** system
 - **Validation** - **checking** that the software is what the customer wants
 - **Evolution** - **changing** the software in response to changing demands.

SOFTWARE PROCESS MODEL

- A simplified representation of a software process, presented from a specific perspective.
- Examples of **process perspectives** are
 - **Workflow** perspective - **sequence** of activities;
 - **Data-flow** perspective - **information** flow;
 - **Role/action** perspective - **who does what**.
- Generic process models
 - **Waterfall**;
 - **Iterative** development;
 - **Component-based** software engineering.

COSTS OF SOFTWARE ENGINEERING

- Roughly **60%** of costs are **development** costs, **40%** are **testing costs**. For **custom software**, evolution costs often exceed development costs.
- **Costs vary** depending on the **type of system** being developed and the **requirements** of system attributes such as **performance** and **system reliability**.
- Distribution of costs depends on the **development model** that is used.

SOFTWARE ENGINEERING METHODS

- Structured approaches to software development which include system models, notations, rules, design advice and process guidance.
- Model descriptions
 - Descriptions of graphical models which should be produced;
- Rules
 - **Constraints** applied to system models;
- Recommendations
 - **Advice** on good design practice;
- Process guidance
 - What activities to **follow**.

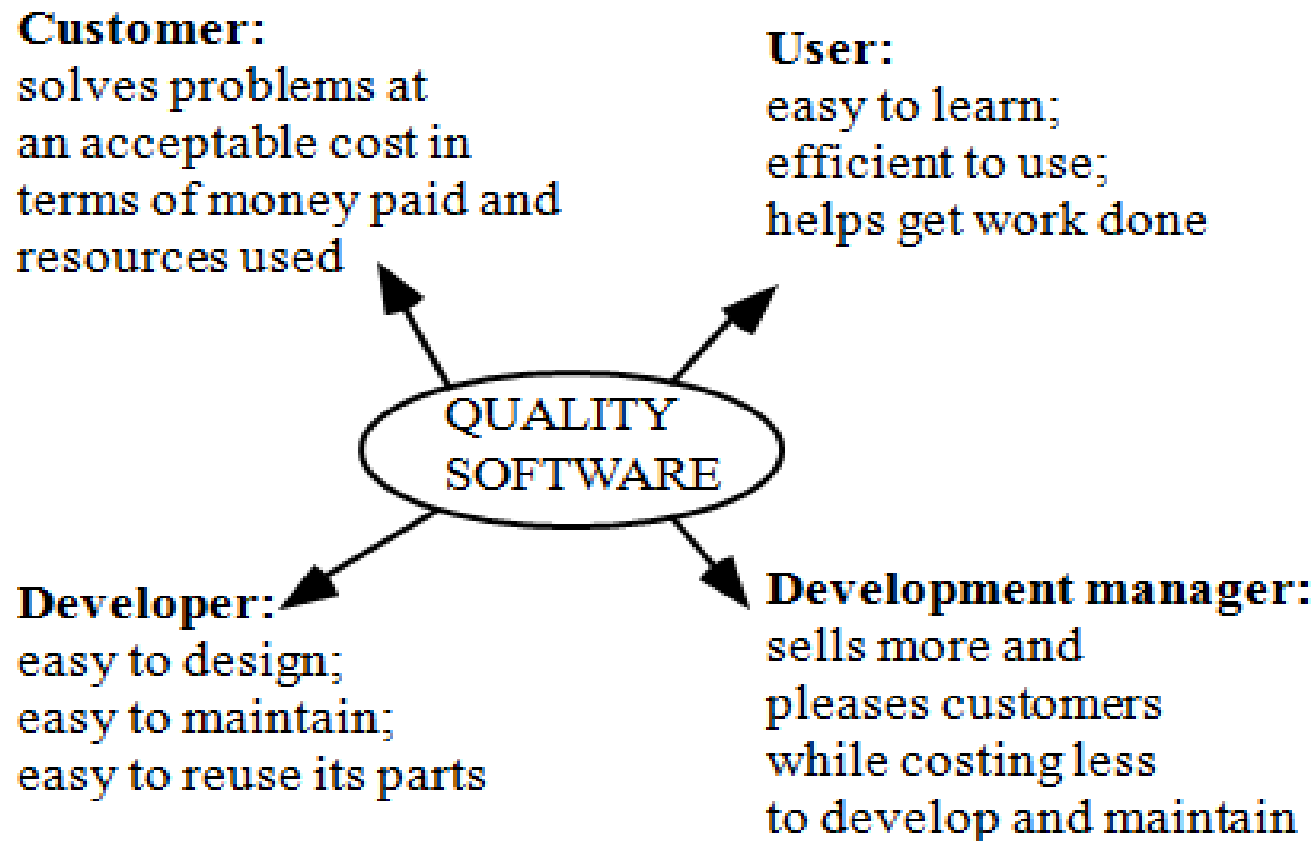
CASE (COMPUTER-AIDED SOFTWARE ENGINEERING)

- Software systems that are intended to provide **automated support for software process activities**.
- CASE systems are often used for **method** support.
- **Upper-CASE**
 - Tools to support the **early process** activities of **requirements** and **design**;
- **Lower-CASE**
 - Tools to support **later** activities such as programming, **debugging** and **testing**.

CHALLENGES FACING SOFTWARE ENGINEERING

- Heterogeneity, delivery and trust.
- Heterogeneity
 - Developing techniques for building software that can cope with heterogeneous platforms and execution environments;
- Delivery
 - Developing techniques that lead to faster delivery of software;
- Trust
 - Developing techniques that demonstrate that software can be trusted by its users.

SOFTWARE QUALITY AND THE STAKEHOLDERS



1. Users

- Those who use the software

2. Customers

- Those who pay for the software

3. Software developers

4. Development Managers

- All four roles can be fulfilled by the same person

Example - ATM stakeholders

- Bank customers
- Representatives of other banks
- Bank managers
- Counter staff
- Database administrators
- Security managers
- Marketing department
- Hardware and software maintenance engineers
- Banking regulators

SOFTWARE QUALITY: CONFLICTS AND OBJECTIVES

- The different qualities can conflict
 - Increasing **efficiency** can **reduce maintainability** or **reusability**
 - Increasing **usability** can reduce **efficiency**
- Setting objectives for quality is a key engineering activity
 - You then design to meet the objectives
 - Avoids 'over-engineering' which wastes money
- **Optimizing** is also sometimes necessary
 - E.g. obtain the highest possible reliability using a fixed budget

INTERNAL QUALITY CRITERIA

- **Internal quality** refers to Characterize *aspects of the **design** and **source** code* of the software
 - Have an effect on the **external** quality attributes
 - E.g.
 - The amount of **commenting** of the code
 - The **complexity** of the code

SHORT TERM VS. LONG TERM QUALITY

○ Short term:

- Does the software *meet the customer's immediate needs*?
- Is it **sufficiently** efficient for the **volume** of data we have *today*?

○ Long term:

- **Maintainability**
- Customer's future needs
- **Scalability**: Can the software **handle** larger volumes of data?

PROFESSIONAL & ETHICAL RESPONSIBILITY

- Software engineers must **accept** that their job involves wider responsibilities than simply the application of technical skills.
- Their work is carried out within legal and social framework.
- They do not use their skills and abilities to behave in a dishonest way or in a way that will bring disrepute to the software engineering profession.

- Some issues of professional responsibilities are:
 - **Confidentiality** – respect the confidentiality of their employers
 - **Competence** – not knowingly accept work which is out with their competence
 - **Intellectual property rights** – be careful to ensure that the intellectual property of employers and clients is protected
 - **Computer misuse** – should not use their technical skills to misuse other people's computers.

- Software engineers shall adhere to the following code of ethics and professional principles
 - Act consistently with public interest
 - Act in the best interests of their clients
 - Ensure their products meet the highest professional standards
 - Maintain integrity and independence in their professional judgment
 - Promote an ethical approach to the management of s/w development & maintenance
 - Advance the integrity and reputation of the profession consistent with the public interest
 - Be fair and supportive to colleagues
 - Participate in lifelong learning

SUMMARY

- Software engineering is an engineering discipline which is concerned with all aspects of software production.
- Software **products** consist of **developed programs** and associated **documentation**.
- Software process consists of **activities** which are involved in developing software product
- **Methods are organized ways of producing software**
- CASE tools are software systems which are designed to support routine **activities** in the software process
- Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.

○ Thank you for your Attention